

CONTEXT ENGINEERING

컨텍스트 엔지니어링

AI 에이전트를 위한 효과적 컨텍스트 설계

Effective Context Engineering for AI Agents



Context와 Context Engineering, 그 어원

CONTEXT · 어원

Context = 맥락(脈絡)

con- + texere → contexere
함께 + 엮다·짜다 → 함께 엮다

脈 + 絡 → 脈絡
잇는 줄기 + 엮다·잇다 → 잇닿는 줄기

영어 **context**(라틴어 contexere)와 한국어 **맥락**(脈絡) — 두 언어가 모두 "엮어 잇는 것"이라는 같은 직관을 담고 있다. (text-textile-texture와 동일 어원)

핵심 · 맥락은 "정보 묶음"이 아니라 의미를 만드는 **주변의 짜임새**다.

ENGINEERING · 어원

Engineering

ingenium → ingeniare → engineer
재주·고안 → 고안하다 → 고안하는 사람

engineering ≠ "우연히 잘 되었다"
그 활동 = **단발적 묘안**

Engineering은 라틴어 **ingenium**(재주·발명)에서 출발했지만, 오늘날엔 단순한 기술이 아닌 **측정·검증·반복 가능한 체계적 설계**를 뜻한다.

핵심 · "잘 되었으면 좋겠다"가 아니라 "**매번 잘 되도록 만든다**"의 차이.

CONTEXT ENGINEERING · 통합 정의

흩어진 정보를 **의도적으로 엮어내어**, LLM이 작업을 **매번 신뢰할 수 있게** 풀어내도록 만드는 체계적 설계

"The discipline of providing all the right context so that an LLM can reliably accomplish the task." — adapted from Philipp Schmid

Context(엮어 잇는 것) + **Engineering**(체계적으로 만들기) = 흩어진 정보를 매번 같은 품질로 직조해 모델에 건네는 일

왜 지금 컨텍스트 엔지니어링인가

PAST

프롬프트 엔지니어링 시대

단일 쿼리(prompt)를 잘 작성하는 능력에 집중하던 시기.

- ▶ 한 번의 입력 → 한 번의 응답
- ▶ 지침 표현 최적화가 핵심
- ▶ 주로 채팅형 단발성 상호작용

NOW

컨텍스트 엔지니어링 시대

다양한 컨텍스트 중 정말 필요한 것만 큐레이션하는 능력이 핵심.

- ▶ 지침 + 도구 + 메모리 + RAG + 히스토리
- ▶ 매 단계마다 무엇을 전달할지 설계
- ▶ 에이전트 성공·실패의 결정 요인

에이전트가 더 오랫동안, 더 많은 도구를 사용할수록 컨텍스트는 길어진다 → 효과적 제어가 핵심 역량이 된다.

Prompt Engineering vs Context Engineering

P Prompt Engineering

"단일 쿼리를 어떻게 잘 작성할 것인가"

- 지침 최적화에 집중
- 일회성(discrete) 작업
- 좋은 표현·문장 구조 탐색
- 주로 채팅형 단발성 상호작용

C Context Engineering

"LLM 추론에 들어갈 **최적의 토큰 집합**을 큐레이션한다"

- 지침 + 도구 + 외부 정보 + 메모리 + 히스토리
- 반복적(iterative) 큐레이션
- 매 단계마다 무엇을 전달할지 설계
- 에이전트 워크플로우 전반

Prompt Engineering $\subset$ Context Engineering · 컨텍스트 엔지니어링은 **시스템 전체를 설계**하는 일이다.

컨텍스트 엔지니어링의 6가지 구성요소

"LLM이 작업을 신뢰할 수 있게 풀 수 있도록 모든 컨텍스트를 제공하는 예술" — Philipp Schmid



01

지침과 규칙

INSTRUCTIONS

시스템 프롬프트, 역할 정의, 정책. 에이전트의 행동 경계를 설정한다.



02

대화 이력

CONVERSATION HISTORY

이전 턴의 사용자 입력과 모델 응답. 단기 컨텍스트의 핵심 자원.



03

장기 기억

LONG-TERM MEMORY

세션 간 영속되는 사용자 프로필·선호·과거 결정. NOTES.md, Memory Tool 등.



04

외부 정보 (RAG)

RETRIEVED CONTEXT

문서·DB·웹에서 검색해온 도메인 지식. 모델 학습 시점 이후의 사실도 포함.



05

사용 가능한 도구

TOOLS

에이전트가 호출할 수 있는 함수·API·MCP. 외부 환경과의 인터페이스.



06

구조화된 출력

STRUCTURED OUTPUT

JSON 스키마·타입 정의 등 응답 형식. 다음 단계 시스템과의 연결을 보장.

저품질 에이전트는 **요청만 받아 기계적으로 응답**하지만, 고품질 에이전트는 위 6가지를 통합해 **상황 맞춤형 응답**을 만든다.

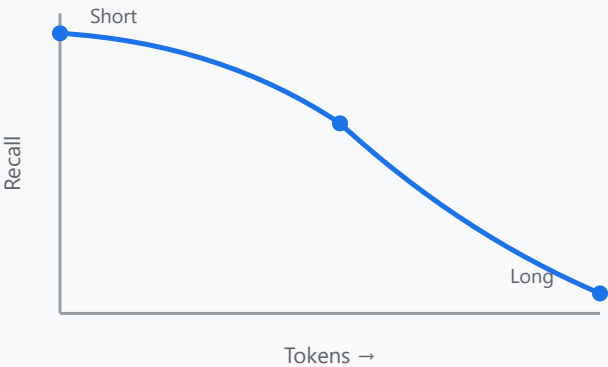
Context Rot — 컨텍스트가 길수록 모델은 흐려진다

왜 "더 큰 컨텍스트 윈도우"가 만능 해결책이 아닐까?

"토큰 수가 증가할수록, 모델이 그 컨텍스트에서 정보를 정확히 **찾아내는** 능력은 **감소**한다."

- 1 **인간의 작업기억과 닮음** — 정보가 많아질수록 집중력이 분산되고, 핵심 신호를 놓치게 된다.
- 2 **학습 데이터 편향** — LLM은 짧은 컨텍스트에 더 잘 반응하도록 훈련됨. 긴 컨텍스트는 본질적으로 약하다.
- 3 **위치 인코딩 보간 한계** — 기술적 확장이 가능하지만 여전히 정밀도 손실이 발생한다.

컨텍스트 길이 vs 정보 인출 정확도 (Recall)



트랜스포머의 n^2 토큰 관계 → 토큰이 많을수록 **각 토큰에 할당되는 어텐션**이 줄어든다 (Attention Dilution).

어텐션 분산 — Attention Dilution

Self-attention의 $O(n^2)$ 구조에서 비롯되는 긴 컨텍스트의 본질적 한계



트랜스포머의 구조

$O(n^2)$

모든 토큰은 다른 모든 토큰과 **쌍별 관계**를 계산. 컨텍스트가 길어질수록 계산-집중 비용이 제곱으로 증가한다.



어텐션 분산

attention dilution

어텐션은 **모든 토큰 쌍에 분산**되므로, 토큰이 많아질수록 각 쌍에 할당되는 어텐션 비중은 얕아진다.



결과: 정밀도 저하

recall ↓

단순히 더 큰 윈도우를 쓰는 것으로는 부족하다. **고신호 토큰만** 선별해서 넣어야 한다.

핵심: 어텐션은 **분산되는 유한 자원**. 따라서 컨텍스트는 양보다 신호 밀도가 중요하다. — Liu et al. (Lost in the Middle, 2023) · Anthropic Engineering

효과적 컨텍스트의 핵심 원칙

THE GUIDING PRINCIPLE

**원하는 결과의 가능성을 최대화하는
가장 작은 고신호 토큰 집합을 찾아라.**

*"Find the smallest possible set of high-signal tokens
that maximize the likelihood of your desired outcome."*

▶ Less is More

▶ High Signal-to-Noise

▶ Curated, not Dumped

시스템 프롬프트의 "적절한 고도" (Right Altitude)

너무 구체적이지도, 너무 추상적이지도 않은 균형점



⚠ TOO LOW

if-else 하드코딩
모든 경우를 명시 → 다양한 케이스에 대응 불가. 깨지기 쉽고 유지보수 비용 폭증.

✓ JUST RIGHT

강력한 휴리스틱 제공
최소한의 명확한 지침으로 시작 → 실패 케이스에만 명령·예제 추가. XML/마크다운으로 섹션 구분.

⚠ TOO HIGH

지나치게 일반적
"잘 해줘" 수준의 추상화 → LLM이 추정해야 할 것이 너무 많다. 일관성 없는 결과.

Tools 설계 — 토큰 효율적인 도구 만들기

에이전트와 외부 세계의 인터페이스, 잘 정의된 함수처럼

PRINCIPLE 01



자기 완결적이고 명확

도구는 **self-contained**하고 오류에 강건해야 한다. 사용법이 명확하고 모호한 의사결정 지점이 없어야 한다.

PRINCIPLE 02



기능 중복 최소화

"인간이 어떤 도구를 쓸지 헷갈리면, AI도 헷갈린다." 비슷한 도구를 여러 개 두지 마라.

PRINCIPLE 03



토큰 효율적 반환

결과를 압축적으로 반환하고, 에이전트가 다음 단계에서 **토큰을 절약하도록 유도**하는 형식으로 응답.

PRINCIPLE 04



Few-shot 예제 포함

옛지 케이스 나열보다 **다양한 정전 (canonical) 예제**로 기대 동작을 보여라. Idempotent 특성도 보장.

가장 흔한 실수: 하나의 도구가 너무 많은 일을 한다 · 또는 너무 많은 도구가 비슷한 일을 한다.

프로덕션 AI 에이전트를 위한 6가지 원칙

"마법 같은 프롬프트 트릭보다 명확한 시스템 설계와 체계적 검증이 중요하다" — app.build

1 명확하고 모순 없는 프롬프트

직접적·구체적 지침이 핵심. 고정 부분과 동적 부분 분리로 **프롬프트 캐싱** 최적화.

prompt caching

2 Lean한 컨텍스트 관리

최소 정보 먼저, 필요할 때 도구로 추가 조회. **관심사 분리**로 효율성 확보.

minimal upfront

3 도구 설계 원칙

사람용 API보다 더 단순하게. 1~3개 파라미터, 다기능 소수 도구, **Idempotent** 보장.

idempotent · few params

4 피드백 루프와 자동 검증

LLM 창의력 + 전통적 검증(컴파일러·테스트) 결합. **Actor-Critic 구조**로 생성과 검증 분리.

actor-critic

5 복구·오류 처리 전략

가드레일로 잘못된 결과 교정·재시도. **유망한 분기는 확장**, 실패는 신속 폐기.

guardrails

6 오류 분석·지속 개선

대부분의 문제는 LLM 성능이 아닌 **시스템 오류**(도구 미설정·프롬프트 불명확). 로그 분석으로 개선.

log-driven

"완벽함보다 신뢰도, 복구 가능성, 반복적 개선에 집중하는 것이 성공의 열쇠"

JIT 컨텍스트 인출 — Just-in-Time Retrieval

모든 데이터를 미리 넣지 말고, 필요한 순간에 동적으로 가져온다

1 전략 비교

✗ PRE-LOAD (구식)

관련 가능성 있는 모든 데이터를 사전에 컨텍스트에 적재 → 윈도우 폭발, 노이즈 증가

✓ JUST-IN-TIME (권장)

가벼운 식별자(파일 경로, URL)만 유지하고, 필요할 때 도구로 동적 로드

인간의 인지와 같다 — 모든 걸 기억하지 않고, 외부 기억과 키워드로 인덱싱한다.

2 Claude Code의 실제 동작

```
# 전체 데이터를 컨텍스트에 로드하지 않음
bash("grep -rn 'TODO' src/") # 표적 쿼리
read("src/api/auth.py", lines="40-80")

# 메타데이터만으로 점진적 발견
glob("**/*.test.ts") → 파일 경로 목록만
read(필요한 파일만 선택적으로)
```

메타데이터가 신호가 된다

- ▶ 폴더 구조 — 책임 영역
- ▶ 네이밍 컨벤션 — 의미 추론
- ▶ 타임스탬프 — 최신성
- ▶ 파일 크기·확장자 — 적재 비용 판단

압축 (Compaction)

컨텍스트 한계에 도달하기 전, 메시지 히스토리를 요약하여 새 윈도우로 이전

동작 흐름



Claude Code 적용 예 — 메시지 히스토리를 LLM에게 넘겨 아키텍처 의사결정·미해결 버그·구현 디테일만 보존.

무엇을 남기고 무엇을 버릴까

KEEP — 보존

- ✓ 아키텍처 의사결정
- ✓ 해결되지 않은 버그
- ✓ 구현 디테일·제약
- ✓ 현재 작업의 목표

DISCARD — 폐기

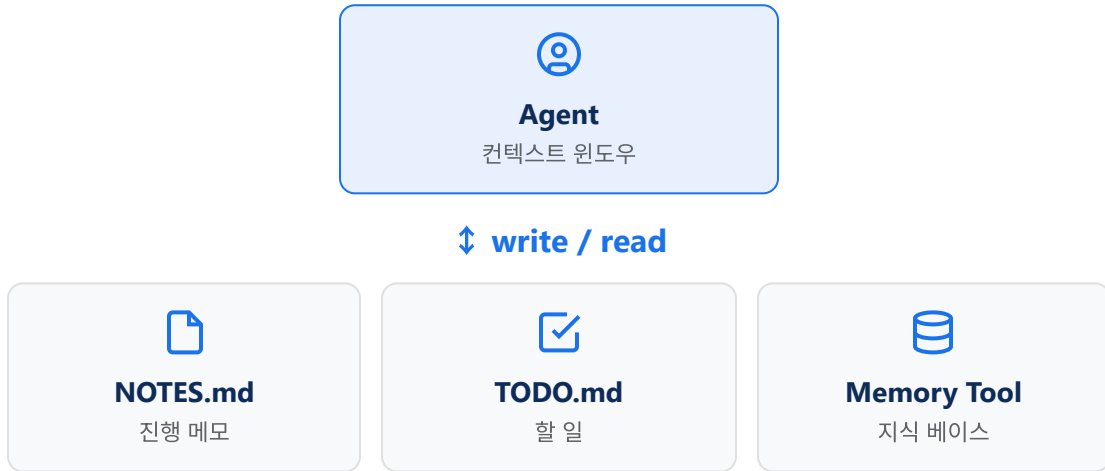
- ✗ 오래된 도구 호출
- ✗ 중간 단계 출력물
- ✗ 이미 해결된 디버깅
- ✗ 중복된 파일 내용

주의: 너무 공격적인 압축은 미묘하지만 결정적인 컨텍스트를 잃을 위험. 균형이 핵심.

구조화된 노트테이킹 (Structured Note-Taking)

컨텍스트 윈도우 바깥(파일 시스템)에 영속적 메모를 남기고 필요할 때 가져온다 — Agentic Memory

아키텍처



```
# 세션 1: 메모 작성
write("NOTES.md", "DB 스키마: users.email UNIQUE...")

# 세션 2 (며칠 뒤): 필요할 때 인출
read("NOTES.md") → 컨텍스트에 적시 적재
```

실제 패턴

To-do 리스트

Claude Code의 TaskList — 진행 상태를 외부에 영속화

NOTES.md 패턴

에이전트가 발견한 핵심 사실·결정·제약을 누적 기록

Memory Tool (Beta)

Sonnet 4.5+ — 세션 간 지식 베이스 자동 관리

핵심 효과 — 컨텍스트 윈도우 비용은 최소화하면서, 영속적 기억을 확보한다.

서브에이전트 아키텍처 (Sub-agent Architecture)

메인은 조율, 서브는 깊이 — 깨끗한 컨텍스트 윈도우로 작업을 분리

분업 구조



왜 효과적인가

- 1 컨텍스트 격리** — 서브가 토큰을 많이 써도 메인은 영향받지 않는다.
- 2 병렬화** — 독립된 서브 작업을 동시에 실행 가능.
- 3 전문화** — 각 서브에게 좁은 시스템 프롬프트·도구만 부여.
- 4 요약된 반환** — 보통 1,000~2,000 토큰의 결과물만 메인으로.

기법 선택 가이드 — 작업 특성에 맞춰 조합하라

압축 · 노트테이킹 · 서브에이전트는 배타적 선택이 아니다. 작업 유형에 맞는 도구를 골라 쓰자.



TECHNIQUE 1

압축 (Compaction)

BEST FOR

광범위한 양방향 소통이 필요한 작업에서 대화 흐름을 유지

예시 · 긴 페어 프로그래밍 세션, 챗봇 상담, 디버깅 대화



TECHNIQUE 2

구조화된 노트테이킹

BEST FOR

명확한 이정표가 있는 작업을 이터레이션을 돌며 개발

예시 · 다단계 빌드, 점진적 리팩토링, 멀티세션 프로젝트



TECHNIQUE 3

서브에이전트 아키텍처

BEST FOR

병렬 탐색이 효과를 발휘하는 복잡한 연구·분석

예시 · 멀티소스 리서치, 대규모 코드베이스 감사, 비교 분석

세 기법은 **중첩하여 사용 가능**하다 — 서브에이전트가 노트를 남기고 메인이 압축하는 식의 조합도 흔하다.

컨텍스트 엔지니어링의 4가지 핵심 축

정보 관리 프레임워크 — Write · Select · Compress · Isolate

1



WRITE

컨텍스트 작성

필요한 정보를 외부 저장소(파일·DB·메모)에 기록하여 영속화한다.

NOTES.md / Memory

2



SELECT

컨텍스트 선택

현재 작업에 정말 필요한 정보만 선별·인출한다 (JIT 전략).

RAG / glob / grep

3



COMPRESS

컨텍스트 압축

긴 히스토리를 요약하여 토큰을 최적화하고 핵심만 보존한다.

Compaction / Summary

4



ISOLATE

컨텍스트 분리

서브에이전트로 작업을 격리·분할하여 메인 컨텍스트를 보호한다.

Sub-agents

개발자의 역할이 "즉흥 코더"에서 "컨텍스트 설계자"로 진화한다 — Stan Polu (Dust)

실전 적용: 좋은 SPEC + agent-skills

에이전트에게 컨텍스트를 잘 주는 두 가지 실전 — 스펙 문서화 + 워크플로우 스킬 패키징

SPEC.md 6가지 핵심 영역

Addy Osmani — GitHub 2,500개 에이전트 파일 분석 결과

Commands

npm test, pytest -v 등 전체 명령어

Structure

src/, tests/, docs/ 폴더 구분

Git Workflow

브랜치 커밋 메시지 규칙

Testing

프레임워크 위치 커버리지

Code Style

실제 스니펫으로 가이드 제시

Boundaries

절대 금지 영역 (시크릿 등)

3단계 경계 시스템

✓ **Always do** — 테스트 실행, 스타일 준수

? **Ask first** — DB 스키마, 의존성 추가

✗ **Never do** — 시크릿 커밋, vendor 편집

agent-skills — 7개 슬래시 커맨드

개발 생명주기 전체를 구조화한 스킬 패키지 (Addy Osmani)

/spec

사양 정의 — "코드 전에 스펙"

/plan

구현 계획 — "소규모 아토믹 태스크로"

/build

점진적 구현 — "한 번에 하나의 슬라이스"

/test

동작 증명 — "테스트는 증거"

/review

품질 게이트 — "코드 헬스 개선"

/code-simplify

단순화 — "영리함보다 명료함"

/ship

프로덕션 배포 — "빠를수록 더 안전함"

핵심 통찰 — 스펙은 에이전트의 "코치이자 심판". **지시의 저주**를 피하려면 한 번에 하나의 태스크로 분할하라.

결론 — 컨텍스트는 유한한 자원이다

KEY TAKEAWAYS

- 1 패러다임 전환: 프롬프트 작성 능력 → **컨텍스트 설계 능력**이 핵심 역량이 되었다.
- 2 어텐션은 분산된다: 더 큰 윈도우보다 **고신호 토큰 선별**이 더 큰 성능 차이를 만든다.
- 3 실천 도구함: **JIT 인출 · 압축 · 노트테이킹 · 서브에이전트** — 작업 특성에 맞춰 조합하라.
- 4 설계의 시작: 시스템 프롬프트는 "**적절한 고도**"로, 도구는 **중복 없이 명확하게**.

THE GUIDING PRINCIPLE

**원하는 결과의 가능성을 최대화하는
가장 작은 고신호 토큰 집합을 찾아라.**

"Find the smallest set of high-signal tokens that maximize the likelihood of your desired outcome."

— Anthropic Engineering

참고자료

본 강의에서 참조한 1차 자료와 한국어 요약 자료 — 카드를 클릭하면 원문이 열립니다

PRIMARY · ANTHROPIC

Effective Context Engineering for AI Agents

Anthropic Engineering Blog

anthropic.com/engineering/effective-context-engineering-for-ai-agents

한국어 요약 · STDY

[요약 번역] AI 에이전트를 위한 효과적인 컨텍스트 엔지니어링

배휘동 — Anthropic 원문 의역

stdy.blog/korean-summary-effective-context-engineering-for-ai-agents

CONCEPT · PHILSCHMID

The New Skill in AI: Context Engineering

Philipp Schmid

philschmid.de/context-engineering

PRINCIPLES · APP.BUILD

Six Principles for Production AI Agents

app.build Engineering Blog

app.build/blog/six-principles-production-ai-agents

SPEC · ADDY OSMANI

Good Spec — 에이전트를 위한 스펙 작성법

Addy Osmani (Google Cloud AI)

addyosmani.com/blog/good-spec/

SKILLS · GITHUB

agent-skills — 엔지니어링 스킬 모음

7개 슬래시 커맨드 + 19개 스킬

github.com/addyosmani/agent-skills

VIBE · BLOGBYASH

바이브 코딩을 위한 컨텍스트 엔지니어링 4축

Stan Polu (Dust) 인터뷰 · 한글

blogbyash.com/translation/vibe-coding-needs-context-engineering/

RESEARCH · CHROMA

Context Rot — 토큰 수와 모델 성능

Chroma Research

research.trychroma.com/context-rot